TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

Unidad 6. Mantenimiento de una Base de Datos

Semana 10

Material teórico para el aula

Proyecto integrador: Food Store | PostgreSQL 16+

Material preparado para trabajo académico y laboratorio. Los ejemplos de comandos se ejecutan primero sobre entornos de prueba y bases descartables.

Propósito de la unidad

La continuidad de una base de datos no depende sólo de un buen diseño. Una base puede estar correctamente normalizada, tener índices adecuados y conservar la integridad de sus relaciones, pero aun así quedar fuera de servicio por una falla de disco, un error humano, una tarea de mantenimiento omitida o un respaldo que nunca fue probado. Esta unidad aborda el mantenimiento como un proceso continuo que combina prevención, observación, automatización y recuperación.

Caso de referencia: Food Store
El proyecto trabaja con las tablas categoria, cliente, producto, pedido y detalle_pedido. Durante la unidad se considera que Food Store ya se encuentra operativa y que conserva información que no debe perderse, como clientes, productos, stock, pedidos y precios históricos de cada detalle. El objetivo deja de ser solamente construir la base: pasa a ser mantenerla disponible y recuperable.

Resultados de aprendizaje

- Distinguir las principales estrategias de respaldo disponibles en PostgreSQL y seleccionar una alternativa adecuada para Food Store.
- Definir objetivos de recuperación mediante RPO y RTO.
- Crear y probar respaldos lógicos con pg_dump y pg_restore en un entorno descartable.
- Reconocer el propósito de VACUUM, ANALYZE, autovacuum y las verificaciones de integridad.
- Consultar métricas de salud mediante las vistas estadísticas de PostgreSQL 16+.
- Organizar tareas periódicas de mantenimiento y registrar su resultado.
- Utilizar OpenCode para producir scripts bajo una especificación explícita, revisar cada línea y probarlos antes de programar su ejecución.
- Ejecutar un simulacro de desastre, medir el tiempo de recuperación y documentar el incidente.

1. El mantenimiento como ciclo operativo

El mantenimiento reúne tareas que se repiten durante toda la vida útil de la base. No se trata de una acción aislada ejecutada cuando aparece un problema. Un plan estable combina respaldos, mantenimiento interno, monitoreo, pruebas de recuperación y registro de evidencias.

Etapa	Pregunta que responde	Aplicación en Food Store
Respaldar	¿Existe una copia utilizable si la base se pierde?	Generar copias periódicas de foodstore y conservar su fecha, tamaño y resultado.
Mantener	¿Las estructuras internas y las estadísticas se conservan en condiciones adecuadas?	Controlar autovacuum, tuplas muertas y estadísticas del planificador.
Monitorear	¿La base muestra señales tempranas de degradación?	Observar conexiones, consultas prolongadas, bloqueos, I/O y crecimiento.
Recuperar	¿La copia puede devolver el servicio en un tiempo aceptable?	Restaurar en foodstore_restore_test y validar datos.
Documentar	¿Queda evidencia de lo realizado y de los incidentes?	Guardar logs, fecha de respaldo, duración, validaciones y conclusiones.

Principio operativo
La existencia de un archivo de respaldo no demuestra que la recuperación sea posible. La confianza aparece cuando la copia se restaura, se valida y se repite el procedimiento con una frecuencia definida.

2. Respaldo y recuperación

PostgreSQL ofrece estrategias de respaldo con alcances diferentes. La selección depende del volumen de datos, el tiempo permitido para recuperar el servicio, la cantidad de información que puede perderse y la complejidad operativa que la organización puede sostener.

2.1. Conceptos básicos

Concepto	Definición aplicada
Respaldo	Copia de información que permite reconstruir total o parcialmente el estado de la base.
Restauración	Proceso que utiliza una copia de respaldo para volver a crear la base o sus objetos.
RPO (Recovery Point Objective)	Cantidad máxima de información reciente que se acepta perder. Se expresa como un intervalo de tiempo.
RTO (Recovery Time Objective)	Tiempo máximo esperado para recuperar el servicio después de una interrupción.
Retención	Período durante el cual se conservan las copias antes de eliminarlas o archivarlas.
Prueba de restauración	Ejecución controlada que verifica que el respaldo puede utilizarse y que los datos restaurados son coherentes.

Ejemplo Food Store
Si se realiza un respaldo cada 24 horas, una falla ocurrida poco antes del siguiente respaldo podría implicar casi un día de datos no incluidos en la última copia. Ese intervalo aproxima el RPO de la estrategia. Si la restauración completa tarda 12 minutos y el servicio debe volver en menos de 30 minutos, el RTO previsto se cumple en ese ensayo.

2.2. Estrategias de respaldo en PostgreSQL

Estrategia	Herramienta o mecanismo	Ventajas	Uso en la unidad
Respaldo lógico	pg_dump / pg_dumpall	Portable, selectivo y sencillo de inspeccionar o restaurar.	Estrategia principal del laboratorio.
Respaldo físico del clúster	pg_basebackup	Copia física del clúster completo; sirve como base para recuperación avanzada y réplicas.	Se estudia conceptualmente.

Estrategia	Herramienta o mecanismo	Ventajas	Uso en la unidad
Archivado continuo + PITR	Base backup + archivos WAL	Permite recuperar el clúster hasta un punto temporal cubierto por los WAL archivados.	Se presenta como estrategia avanzada.

Para Food Store se utiliza un respaldo lógico en formato custom porque el proyecto es acotado, la copia puede almacenarse en un único archivo y pg_restore permite inspeccionar y restaurar el contenido. La estrategia no impide que una instalación real adopte posteriormente respaldo físico y recuperación a un punto en el tiempo.

2.3. Respaldo lógico con pg_dump

pg_dump genera una copia consistente de una sola base de datos y puede ejecutarse mientras otros usuarios continúan leyendo y escribiendo. El formato custom se selecciona con -Fc y se restaura con pg_restore.

Ejemplo de respaldo en formato custom

```
pg_dump -h localhost -p 5432 -U postgres -d foodstore -Fc -f backups/foodstore_2026-10-05_2300.dump
```

- -h localhost identifica el servidor.
- -p 5432 indica el puerto utilizado por PostgreSQL.
- -U postgres establece el rol con el que se realiza la conexión.
- -d foodstore selecciona la base que se respalda.
- -Fc utiliza el formato custom.
- -f define el archivo de salida.

Credenciales
La contraseña no se incorpora al script ni al repositorio. En Windows puede utilizarse el archivo %APPDATA%\postgresql\pgpass.conf; en sistemas Unix puede utilizarse ~/.pgpass. Este mecanismo permite automatizar conexiones sin dejar la contraseña escrita en el código del mantenimiento.

2.4. Verificación inicial del archivo

La creación del archivo constituye sólo el primer control. El respaldo debe finalizar sin error, conservar un tamaño razonable y permitir que pg_restore lea su catálogo.

Inspección del contenido del respaldo

```
pg_restore --list backups/foodstore_2026-10-05_2300.dump
```

El listado no reemplaza a la restauración, pero detecta de forma temprana archivos que no pueden interpretarse como un archivo válido de pg_dump.

2.5. Restauración en un entorno descartable

La prueba se realiza sobre una base independiente. De esta manera, el respaldo se valida sin modificar foodstore ni mezclar datos de prueba con datos de trabajo.

Restauración controlada

```
createdb -h localhost -p 5432 -U postgres foodstore_restore_test

pg_restore -h localhost -p 5432 -U postgres -d foodstore_restore_test --no-owner --no-privileges
backups/foodstore_2026-10-05_2300.dump
```

Regla de seguridad
El simulacro no restaura sobre la base original. La base foodstore_restore_test puede eliminarse al terminar la validación. Las operaciones destructivas se reservan para entornos descartables y se revisan antes de ejecutarse.

2.6. Validación posterior a la restauración

Una restauración correcta debe comprobar algo más que la ausencia de mensajes de error. El equipo verifica que las tablas existan, que las cantidades de filas sean razonables y que las restricciones principales sigan vigentes.

Controles mínimos de Food Store

```
SELECT 'categoria' AS tabla, COUNT(*) AS filas FROM categoria
UNION ALL SELECT 'cliente', COUNT(*) FROM cliente
UNION ALL SELECT 'producto', COUNT(*) FROM producto
UNION ALL SELECT 'pedido', COUNT(*) FROM pedido
UNION ALL SELECT 'detalle_pedido', COUNT(*) FROM detalle_pedido;

SELECT COUNT(*) AS productos_invalidos
FROM producto
WHERE precio < 0 OR stock < 0;

SELECT COUNT(*) AS detalles_invalidos
FROM detalle_pedido
WHERE cantidad <= 0 OR precio_unitario < 0;
```

La comparación más útil registra estas mismas cantidades antes del respaldo y después de la restauración. El resultado esperado para los controles de valores inválidos es cero.

3. RPO y RTO: convertir la recuperación en un objetivo medible

Una estrategia de respaldo necesita objetivos explícitos. RPO y RTO permiten discutir el riesgo con medidas concretas en lugar de expresiones imprecisas como “hacer copias seguido” o “recuperar rápido”.

Escenario	Frecuencia de respaldo	RPO aproximado	RTO de prueba	Lectura
Laboratorio académico	1 respaldo diario	Hasta 24 h	Menor a 20 min	Adecuado para practicar el procedimiento completo.
Tienda con alta actividad	Cada 1 h	Hasta 1 h	Menor a 30 min	Reduce pérdida potencial, pero aumenta frecuencia y administración.

Escenario	Frecuencia de respaldo	RPO aproximado	RTO de prueba	Lectura
Servicio crítico con PITR	Base backup + WAL continuo	Minutos o segundos según política	Depende de infraestructura y procedimiento	Requiere una estrategia física y gestión de WAL.

Interpretación
RPO no indica cuánto tarda la restauración. RTO no indica cuántos datos se pierden. Ambos objetivos se complementan y deben comprobarse mediante pruebas periódicas.

4. Tareas programadas de mantenimiento

PostgreSQL realiza parte del mantenimiento de manera automática, pero el administrador conserva la responsabilidad de verificar su funcionamiento, decidir cuándo corresponde una intervención adicional y registrar resultados. La programación periódica evita que tareas necesarias dependan de la memoria de una persona.

4.1. VACUUM y autovacuum

PostgreSQL utiliza MVCC. Una actualización o eliminación puede dejar versiones de filas que ya no son visibles para las transacciones normales. VACUUM permite recuperar ese espacio para reutilización interna y realiza tareas necesarias para la salud de las tablas. El proceso autovacuum automatiza gran parte de este trabajo.

Control del mantenimiento automático

```
SELECT relname,
       n_live_tup,
       n_dead_tup,
       last_autovacuum,
       last_autoanalyze
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;
```

VACUUM FULL no es mantenimiento rutinario
VACUUM FULL reescribe la tabla y requiere un bloqueo exclusivo. Por ese motivo no se programa como una tarea frecuente. Se reserva para situaciones específicas en las que la recuperación física de espacio justifica el costo y la indisponibilidad asociada.

4.2. ANALYZE y estadísticas del planificador

ANALYZE recolecta estadísticas sobre la distribución de los datos. El planificador utiliza esa información para estimar costos y seleccionar planes de ejecución. Cuando los datos cambian de forma importante, estadísticas desactualizadas pueden producir decisiones menos convenientes.

Actualización controlada de estadísticas

```
ANALYZE producto;
ANALYZE pedido;
ANALYZE detalle_pedido;

-- En una ventana de mantenimiento también puede utilizarse:
VACUUM (ANALYZE, VERBOSE) producto;
```

4.3. Verificación de integridad

La integridad se controla en distintos niveles. Las restricciones PK, FK, UNIQUE y CHECK protegen reglas declarativas; las consultas de consistencia verifican invariantes del negocio; y las herramientas de PostgreSQL permiten realizar controles físicos adicionales.

Nivel	Ejemplo en Food Store	Herramienta
Declarativo	stock >= 0, precio >= 0, email único, FK pedido-cliente	Constraints del esquema.
Lógico	No existen detalles con cantidad <= 0; las cantidades restauradas coinciden con el origen	Consultas SQL de validación.
Físico	Comprobación de relaciones e índices soportados para detectar corrupción	pg_amcheck.
Clúster	Verificación de checksums si fueron habilitados en el clúster	data_checksums / pg_checksums con condiciones apropiadas.

Ejemplo de comprobación física con pg_amcheck

```
pg_amcheck -h localhost -p 5432 -U postgres foodstore
```

pg_amcheck se utiliza como una verificación adicional. No sustituye a los respaldos ni a la prueba de restauración.

4.4. ¿Cómo se programan las tareas?

El núcleo de PostgreSQL no incorpora un programador general de tareas para ejecutar cualquier script operativo. La programación puede delegarse al sistema operativo, por ejemplo mediante el Programador de tareas de Windows o cron en Linux. También existen extensiones específicas, pero no se asumen como requisito del laboratorio.

Frecuencia de referencia	Tarea	Evidencia esperada
Diaria	Respaldo lógico de foodstore	Archivo con fecha y log de ejecución.
Diaria	Chequeo breve de salud	Archivo o salida con conexiones, tamaño y consultas prolongadas.
Semanal	Revisión de tuplas muertas, autovacuum y estadísticas	Consulta de pg_stat_user_tables y decisión documentada.

Frecuencia de referencia	Tarea	Evidencia esperada
Semanal	Prueba de restauración del respaldo más reciente	Base descartable restaurada y controles de filas.
Mensual	Revisión de retención y espacio utilizado por respaldos	Copias antiguas eliminadas según la política definida.

4.5. Ejemplo de programación

En Windows, el Programador de tareas puede ejecutar el script de PowerShell en un horario definido. La acción puede invocar powershell.exe y pasar como argumento la ruta del archivo backup_foodstore.ps1. En Linux, cron puede ejecutar el script equivalente. En ambos casos, la tarea se considera operativa sólo si deja un log que luego se revisa.

Ejemplo conceptual para Programador de tareas de Windows

```
Programa o script: powershell.exe
Argumentos: -NoProfile -File "C:\FoodStore\scripts\backup_foodstore.ps1"
Iniciar en: C:\FoodStore
```

Ejemplo equivalente con cron: ejecución diaria a las 02:00

```
0 2 * * * /opt/foodstore/scripts/backup_foodstore.sh >> /opt/foodstore/logs/scheduler.log 2>&1
```

5. Monitoreo y métricas de salud

El monitoreo busca detectar desviaciones antes de que se conviertan en fallas visibles. PostgreSQL expone vistas estadísticas que permiten observar actividad, transacciones, tablas, índices e I/O. Una métrica aislada no determina por sí sola que exista un problema; la interpretación requiere contexto, comparación histórica y verificación.

5.1. Conexiones y actividad

Estado general de la base

```
SELECT datname,
       numbackends,
       xact_commit,
       xact_rollback,
       blks_read,
       blks_hit
FROM pg_stat_database
WHERE datname = 'foodstore';
```

Sesiones activas y duración

```
SELECT pid,
       usename,
       state,
       now() - query_start AS duracion,
       query
FROM pg_stat_activity
WHERE datname = 'foodstore'
  AND state <> 'idle'
ORDER BY duracion DESC;
```

Una consulta prolongada no se cancela automáticamente por aparecer en esta lista. Primero se identifica la causa, se revisa el plan, se observa si existe un bloqueo y se evalúa el impacto de cualquier acción.

5.2. Bloqueos

Lectura básica de bloqueos

```
SELECT a.pid,
       a.usename,
       a.state,
       l.locktype,
       l.mode,
       l.granted,
       a.query
FROM pg_locks l
JOIN pg_stat_activity a ON a.pid = l.pid
WHERE a.datname = 'foodstore'
ORDER BY l.granted, a.pid;
```

La presencia de bloqueos es normal en un motor transaccional. El problema aparece cuando una espera se prolonga, bloquea operaciones importantes o forma parte de un patrón repetitivo.

5.3. Salud de tablas e índices

Métricas por tabla

```
SELECT relname,
       seq_scan,
       idx_scan,
       n_live_tup,
       n_dead_tup,
       last_autovacuum,
       last_autoanalyze
FROM pg_stat_user_tables
ORDER BY relname;
```

Uso de índices

```
SELECT relname AS tabla,
       indexrelname AS indice,
       idx_scan
FROM pg_stat_user_indexes
ORDER BY idx_scan ASC, relname;
```

Un índice con pocas lecturas no se elimina de manera automática. Debe considerarse la antigüedad de las estadísticas, la carga real, las consultas poco frecuentes pero críticas y el costo de mantener el índice en escrituras.

5.4. Tamaño y crecimiento

Tamaño de la base y de sus tablas

```
SELECT pg_size_pretty(pg_database_size('foodstore')) AS tamano_base;

SELECT relname,
       pg_size_pretty(pg_total_relation_size(relid)) AS tamano_total
FROM pg_stat_user_tables
ORDER BY pg_total_relation_size(relid) DESC;
```

El crecimiento se interpreta comparando mediciones tomadas en distintos momentos. El tamaño por sí solo no indica una falla, pero un crecimiento inesperado puede justificar revisar cargas, retención de datos, índices y actividad de VACUUM.

5.5. I/O en PostgreSQL 16+

PostgreSQL 16 incorpora la vista pg_stat_io para observar operaciones de entrada y salida agrupadas por tipo de backend, objeto y contexto. Esta información complementa las métricas de tablas y las herramientas del sistema operativo.

Lectura simplificada de pg_stat_io

```
SELECT backend_type,
       object,
       context,
       reads,
       writes,
       fsyncs
FROM pg_stat_io
ORDER BY backend_type, object, context;
```

Sobre los umbrales
No existe un número universal que convierta automáticamente una métrica en “saludable” o “crítica”. El plan define valores de referencia para el entorno, compara tendencias y verifica la causa antes de modificar configuración, cancelar sesiones o reconstruir objetos.

6. Diagnóstico asistido con IA

La IA puede acelerar la lectura de métricas y logs, pero no reemplaza la comprobación técnica. El diagnóstico se formula como una hipótesis que debe contrastarse con consultas, planes de ejecución, logs y comportamiento reproducible.

Paso	Práctica esperada
1. Observar	Se recopila la métrica o el fragmento de log sin modificar todavía el sistema.
2. Formular hipótesis	La herramienta puede proponer causas posibles, pero cada causa se trata como una hipótesis.
3. Verificar	Se ejecutan consultas que confirmen o descarten cada hipótesis.
4. Decidir	Se selecciona una acción sólo cuando la evidencia la justifica.

Paso	Práctica esperada
5. Probar	El cambio se ensaya en un entorno controlado cuando sea posible.
6. Registrar	Se documenta qué se observó, qué se modificó y qué resultado produjo.

Ejemplo de diagnóstico
Si pg_stat_user_tables muestra muchas tuplas muertas en producto, la interpretación no concluye inmediatamente que “VACUUM está roto”. Primero se revisan last_autovacuum, la actividad reciente de UPDATE/DELETE, la configuración de autovacuum y la evolución de la métrica. Sólo después se decide si corresponde esperar al proceso automático, ejecutar un VACUUM manual o investigar la configuración.

7. OpenCode como motor de automatización bajo especificación

OpenCode se utiliza para producir scripts repetibles a partir de requisitos explícitos. El objetivo académico no consiste en aceptar código generado de forma automática, sino en convertir una especificación operativa en archivos que puedan revisarse, ejecutarse en un entorno descartable y programarse con seguridad.

7.1. Qué puede aportar OpenCode

- Leer la estructura del repositorio y las instrucciones persistentes del proyecto.
- Crear scripts de respaldo, mantenimiento, monitoreo y restauración de prueba.
- Generar archivos de log con fecha, resultado y código de salida.
- Revisar consistencia entre una especificación y los scripts producidos.
- Proponer comandos de verificación antes de aplicar una acción.
- Ayudar a documentar un procedimiento reproducible.

La programación temporal no queda a cargo de OpenCode. Una vez validado el script, el sistema operativo o un servicio de planificación se encarga de ejecutarlo a la frecuencia definida.

7.2. Instrucciones persistentes del proyecto

El repositorio puede incluir un archivo AGENTS.md con reglas que OpenCode carga como instrucciones del proyecto. En Food Store conviene definir restricciones operativas que eviten acciones destructivas sobre la base principal.

Ejemplo de instrucciones para el repositorio

```
# AGENTS.md - Food Store / Unidad 6

- PostgreSQL objetivo: 16 o superior.
- La base productiva de referencia se llama foodstore.
- Toda restauración de prueba usa foodstore_restore_test.
- No se permite ejecutar DROP DATABASE foodstore.
- Las contraseñas no se escriben en scripts ni se versionan.
- Todo script debe registrar fecha, operación y resultado.
- Los comandos destructivos requieren revisión humana previa.
- Antes de programar una tarea, el script se prueba manualmente.
- Los archivos generados deben ser legibles y estar comentados.
```

7.3. Trabajo bajo spec

La especificación describe el resultado esperado antes de generar código. De esta manera, el script puede evaluarse contra requisitos concretos y no contra una impresión general.

Ejemplo reducido de especificación

```
# SPEC_U6.md

Objetivo: automatizar el mantenimiento básico de Food Store.

El plan debe incluir:
1. Un respaldo lógico diario en formato custom.
2. Nombre de archivo con fecha y hora.
3. Carpeta de logs separada de la carpeta de respaldos.
4. Ninguna contraseña dentro del script.
5. Verificación del código de salida de pg_dump.
6. health_check.sql con métricas de actividad, tablas y tamaño.
7. maintenance.sql con acciones seguras y justificadas.
8. restore_test que cree foodstore_restore_test y restaure allí.
9. Ningún DROP o modificación sobre foodstore.
10. Mensajes claros cuando una tarea falla.
```

7.4. Flujo de trabajo seguro

1. El equipo define el objetivo, la frecuencia y los límites de seguridad.
2. La especificación se guarda en el repositorio antes de generar los scripts.
3. OpenCode produce una primera versión a partir de la spec y de AGENTS.md.
4. Cada comando se lee línea por línea y se relaciona con un requisito.
5. El script se ejecuta manualmente sobre el entorno de laboratorio.
6. El respaldo se restaura en foodstore_restore_test.
7. Las consultas de validación confirman estructura y datos.
8. Recién después de las pruebas se configura la ejecución programada.
9. Los logs se revisan de forma periódica; una tarea automática que falla sin ser observada no constituye un plan de mantenimiento efectivo.

8. Ejemplo de plan automatizado para Food Store

Componente	Archivo sugerido	Función
Instrucciones	AGENTS.md	Fija reglas del repositorio y límites de seguridad.
Especificación	SPEC_U6.md	Define requisitos verificables del mantenimiento.
Respaldo	backup_foodstore.ps1 o backup_foodstore.sh	Ejecuta pg_dump, genera nombre fechado y registra resultado.
Mantenimiento	maintenance.sql	Consulta estado y ejecuta sólo tareas previamente justificadas.
Monitoreo	health_check.sql	Obtiene métricas de salud reproducibles.
Restauración	restore_test.ps1 o restore_test.sh	Restaura la última copia en una base descartable.
Evidencias	logs/	Conserva fecha, duración y resultado de las ejecuciones.

8.1. Ejemplo de script de respaldo en PowerShell

Modelo didáctico. La ruta de pg_dump puede requerir ajuste según la instalación.

```
$fecha = Get-Date -Format "yyyy-MM-dd_HHmm"
$backupDir = ".\backups"
$logDir = ".\logs"
$archivo = Join-Path $backupDir "foodstore_$fecha.dump"
$log = Join-Path $logDir "backup_$fecha.log"

New-Item -ItemType Directory -Force -Path $backupDir, $logDir | Out-Null

"[$(Get-Date)] Inicio de respaldo" | Out-File $log

pg_dump -h localhost -p 5432 -U postgres -d foodstore -Fc -f $archivo 2>> $log

if ($LASTEXITCODE -eq 0) {
    "[$(Get-Date)] Respaldo OK: $archivo" | Out-File $log -Append
} else {
    "[$(Get-Date)] ERROR: pg_dump finalizo con codigo $LASTEXITCODE" | Out-File $log -Append
    exit $LASTEXITCODE
}
```

El script no contiene una contraseña, crea carpetas si no existen, utiliza un nombre fechable y finaliza con error cuando pg_dump falla. Antes de programarlo, el equipo comprueba manualmente que el archivo pueda restaurarse.

8.2. Ejemplo de chequeo de salud

Archivo SQL de observación

```
-- health_check.sql
SELECT now() AS momento,
       current_database() AS base,
       pg_size_pretty(pg_database_size(current_database())) AS tamano;

SELECT numbackends, xact_commit, xact_rollback, blks_read, blks_hit
FROM pg_stat_database
WHERE datname = current_database();

SELECT relname, n_live_tup, n_dead_tup, last_autovacuum, last_autoanalyze
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;

SELECT pid, state, now() - query_start AS duracion, query
FROM pg_stat_activity
WHERE datname = current_database()
  AND state <> 'idle'
ORDER BY duracion DESC;
```

9. Simulacro de desastre y restauración contra reloj

El simulacro transforma el respaldo en una capacidad demostrable. El objetivo no consiste en generar tensión artificial, sino en medir cuánto tarda el procedimiento, identificar pasos ambiguos y mejorar la documentación antes de enfrentar una falla real.

9.1. Escenario didáctico

Incidente
Se considera que Food Store deja de estar disponible o que una operación accidental compromete datos importantes. La base original no se destruye durante la actividad. El equipo trabaja con el supuesto de pérdida y reconstruye el servicio en foodstore_restore_test utilizando el último respaldo válido.

9.2. Procedimiento

1. Se registra la hora de inicio del incidente.
2. Se identifica el respaldo más reciente y se verifica que pueda listarse con pg_restore --list.
3. Se crea la base foodstore_restore_test.
4. Se restaura el respaldo con pg_restore.
5. Se ejecutan los controles de cantidad de filas y las validaciones de integridad definidas para Food Store.
6. Se registra la hora en que la base restaurada queda validada.
7. Se calcula el tiempo total de recuperación observado y se compara con el RTO previsto.
8. Se estima la posible pérdida de datos según la antigüedad del respaldo y se compara con el RPO.
9. Se documentan errores, pasos demorados y mejoras para la siguiente prueba.

9.3. Informe post-incidente

Campo	Contenido esperado
Incidente simulado	Descripción breve del evento que obliga a restaurar.
Último respaldo disponible	Fecha, hora y nombre del archivo.
RPO previsto y observado	Antigüedad del respaldo respecto del incidente.
RTO previsto y observado	Objetivo y tiempo real desde el inicio hasta la validación.
Validaciones	Tablas recuperadas, cantidades de filas y controles de integridad.
Problemas encontrados	Errores de ruta, permisos, credenciales, comandos o documentación.
Acciones de mejora	Cambios concretos que reducen riesgo o tiempo en la siguiente prueba.

10. Plan de mantenimiento recomendado para el laboratorio

Frecuencia	Acción	Criterio de éxito
Cada día	Generación del respaldo lógico de Food Store.	pg_dump finaliza correctamente, el archivo existe y queda registrado.
Cada día	Chequeo de salud.	Se genera evidencia de actividad, tamaño y estado de tablas.
Cada semana	Revisión de autovacuum y estadísticas.	Se identifican tablas con crecimiento de tuplas muertas o estadísticas antiguas.

Frecuencia	Acción	Criterio de éxito
Cada semana	Restauración de prueba.	La base descartable se reconstruye y supera las validaciones.
Cada mes	Revisión de retención y del procedimiento.	Las copias cumplen la política y la documentación permite repetir la recuperación.

11. Errores frecuentes

Error	Por qué es un problema	Corrección
Guardar la contraseña dentro del script	Expone una credencial en archivos, repositorios o capturas.	Usar el archivo de contraseñas de PostgreSQL o un mecanismo seguro equivalente.
Crear respaldos que nunca se restauran	Un archivo puede estar incompleto, ser incompatible con el procedimiento o no incluir lo esperado.	Programar pruebas de restauración.
Ejecutar DROP sobre la base original durante la práctica	Convierte una actividad de aprendizaje en un riesgo innecesario.	Restaurar en una base descartable con otro nombre.
Programar VACUUM FULL como tarea rutinaria	Puede bloquear tablas durante una operación costosa.	Confiar en autovacuum y justificar cualquier intervención adicional.
Aceptar una explicación de IA sin verificarla	Una hipótesis plausible puede ser incorrecta para el entorno observado.	Comprobar métricas, logs y comandos antes de actuar.
Automatizar sin logs	Una tarea puede fallar durante días sin que nadie lo detecte.	Registrar fecha, resultado y código de salida.

12. Síntesis de la unidad

El mantenimiento de Food Store se organiza como un ciclo: se generan respaldos, se mantiene el motor, se observan métricas, se verifica la integridad y se ensaya la recuperación. La automatización reduce tareas repetitivas, pero no elimina la responsabilidad de revisar resultados. OpenCode aporta velocidad para producir scripts y documentación bajo una spec; PostgreSQL aporta las herramientas de respaldo, estadísticas y mantenimiento; y el sistema operativo ejecuta las tareas en el horario definido. La validez del plan se demuestra cuando un respaldo reciente puede restaurarse y verificarse dentro de los objetivos de recuperación establecidos.

Lista de control antes de considerar operativo el plan

- La base foodstore cuenta con un respaldo reciente y fechado.
- El respaldo puede listarse con pg_restore y restaurarse en foodstore_restore_test.
- Las tablas principales recuperan cantidades de filas coherentes.
- Los controles de stock, precios y detalle no detectan valores inválidos.
- El equipo conoce el RPO y el RTO definidos para el laboratorio.
- Las tareas programadas generan logs y los logs se revisan.
- Autovacuum y estadísticas se observan antes de decidir mantenimiento manual.

- Las métricas se interpretan con contexto y no mediante umbrales arbitrarios.
- Los scripts creados con OpenCode se comparan contra la spec y se prueban manualmente.
- Existe un procedimiento escrito para repetir el simulacro de recuperación.

Referencias oficiales

Las prácticas y comandos de la unidad se apoyan en documentación oficial. Las opciones concretas pueden ajustarse al sistema operativo y a la instalación utilizada en el laboratorio.

1. PostgreSQL 16 - Backup and Restore. https://www.postgresql.org/docs/16/backup.html
2. PostgreSQL 16 - pg_dump. https://www.postgresql.org/docs/16/app-pgdump.html
3. PostgreSQL 16 - pg_restore. https://www.postgresql.org/docs/16/app-pgrestore.html
4. PostgreSQL 16 - Routine Database Maintenance Tasks. https://www.postgresql.org/docs/16/maintenance.html
5. PostgreSQL 16 - VACUUM. https://www.postgresql.org/docs/16/sql-vacuum.html
6. PostgreSQL 16 - ANALYZE. https://www.postgresql.org/docs/16/sql-analyze.html
7. PostgreSQL 16 - Monitoring Database Activity. https://www.postgresql.org/docs/16/monitoring.html
8. PostgreSQL 16 - Cumulative Statistics System. https://www.postgresql.org/docs/16/monitoring-stats.html
9. PostgreSQL 16 - pg_amcheck. https://www.postgresql.org/docs/16/app-pgamcheck.html
10. PostgreSQL 16 - pg_basebackup. https://www.postgresql.org/docs/16/app-pgbasebackup.html
11. PostgreSQL 16 - Continuous Archiving and PITR. https://www.postgresql.org/docs/16/continuous-archiving.html
12. PostgreSQL 16 - Password File. https://www.postgresql.org/docs/16/libpq-pgpass.html
13. OpenCode - Instructions / AGENTS.md. https://opencode.ai/docs/instructions/